\Context: *Troubleshooting Suite (Panacea), launched in 2025, is an end-to-end troubleshooting system which enables automatic detection of issues and perform diagnosis. This PRFAQ builds on the initial foundation and expands the diagnostic and root cause identification capabilities.*

PRESS RELEASE: DS2's Troubleshooting Suite (Panacea) automates Cross-Domain Diagnosis and Root-Cause Attribution Across Devices, Apps, and Services

DENVER, CO — December 1, 2026 — Troubleshooting Suite (Panacea) now delivers automated, cross-domain diagnosis and root-cause attribution for issues across devices, applications, and services. By automatically collecting and correlating diagnostic data spanning logs, crashes, user interactions, system-level traces, and cloud service data — the system surfaces root causes and recommends remediation actions without requiring manual investigation, reducing Mean-Time-to-Resolution (MTTR) from days to hours and improving first-attempt issue resolution.

Before today, when an issue occurred, engineers had to manually reconstruct timelines by piecing together device logs and crashes, user interaction data and service logs across siloed systems with inconsistent schemas and timestamps. Multi-domain issues required deep domain expertise, significant effort, and coordination across multiple points of contact to correlate the right signals before root cause could be identified. Even after initial analysis, engineers had to trace failures back to specific code changes, or configuration updates—making issue reproduction difficult, increasing the MTTR as well as prolonging customer impact through delayed mitigation, degraded experiences, and developer time spent on investigation instead of resolution.

Starting today, when an issue is detected, the system performs end-to-end cross-domain analysis by automatically correlating telemetry across device and service logs, crash stack traces, user interactions, measurement events, system-level traces, customer support data. It synthesizes this data into a unified, time-ordered diagnostic ledger, giving developers a complete view of what led to an issue without having to manually gather details across fragmented systems. Rather than collecting data indiscriminately, the system employs adaptive collection—starting with lightweight diagnostic data such as key logs, crash traces and metadata, and then expanding coverage only when needed to increase confidence in attribution. Domain-specific analyzers run alongside base analyzers against this shared context to identify likely causes of failures. Domain experts can further enrich the diagnostic context by contributing specialized knowledge such as runbooks, code mappings, and domain-specific insights, which are applied against a broader shared, multi-domain context to uncover cross-system patterns and improve attribution confidence. Together, these capabilities create a continuous data enrichment cycle, providing developers with high confidence diagnostic insights for faster remediation.

"Our teams can now solve problems much faster, so customers continue to have delightful experiences with our devices," said Amazon VP. *"Engineers no longer have to look for signals across fragmented systems—the answers come to them, turning days of investigation into minutes"*

Cross-domain diagnostic insights are accessible through a single Troubleshooting AI Agent, which can be accessed via Kiro CLI, Kiro IDE, Loom, or invoked through agent-to-agent integration. Additionally, an interactive device-level diagnostic timeline enables deep-dive analysis. These insights surface failure patterns across multiple domains and the precise sequence of events leading to an issue, enabling teams to reproduce and validate failures earlier during beta and pre-launch phases, improving device quality and release stability. Instead of gathering data, teams now start with answers to what failed, why it failed, which code path caused it, and the remediation steps needed to resolve issue. By transforming fragmented, multi-system investigation into an autonomous, automated workflow, Troubleshooting Suite enables teams to move quickly from detection to resolution, reduce MTTR, and deliver better customer experiences at scale.

To learn more and start automating your investigations, visit the Troubleshooting Suite wiki.

FREQUENTLY ASKED QUESTIONS

Q1. What is your North Star?

Our North Star is to make failures invisible to customers by automating and expediting end-to-end investigation and remediation across Amazon devices, applications and services.

Q2. Who are your primary customers and how does this launch help them?

The primary customers of this system are developers and troubleshooters across Devices and Applications teams responsible for diagnosing and resolving issues across devices, applications, and services. These roles include Software Development Engineers (SDEs), Quality Assurance Engineers (QAEs), Technical Program Managers (TPMs), and Software Development Managers (SDMs).

SDEs can now quickly trace failures to responsible code paths and gain cross-domain insights that previously required deep expertise across multiple disciplines. Senior SDEs contribute as domain experts by authoring analyzers and runbooks that guide root-cause analysis and remediation strategies. QAEs use the same context to reproduce failures, validate fixes, and detect issues earlier during beta and pre-launch testing. TPMs gain clear visibility into the responsible domain and can drive ownership, coordination, and corrective actions across teams. SDMs benefit from aggregated insights into systemic failure patterns, enabling better prioritization of engineering work and improved operational health across the organization.

Q3. Remind me again, what is Troubleshooting Suite (Panacea)?

Troubleshooting Suite is a system for Amazon devices and application teams supporting the entire troubleshooting lifecycle. It enables users to not only track device health status but also proactively detect risks, assess customer impact, and ultimately facilitate root cause analysis (PRFAQ link). In 2025, we launched five issue detection models for OTA, where we continuously monitor device cohorts for issues and automatically generate investigation summary that includes AI generated key findings and insights, automatically pulled device logs, top crashes, and customer journey visualizations to aid in diagnosis.

Q4. What data will the system support at launch?

At launch, the system integrates with the following data sources across multiple domains, that serve two purposes: detecting issues and providing diagnostic data for attribution:

a) Device and application logs — triggered post issue detection
b) Device and application crashes — stack traces and crash metadata
c) User interactions (Panorama) — user activity and engagement signals
d) Cloud service logs (Atocha) — backend service-level events
e) Customer support call data — Customer reported symptoms and call transcripts.
f) Jira and SIM tickets — issue and ownership details along with escalation history
g) Measurement and diagnostic events — device metrics and events
h) System activities — network/function calls, memory/storage events for low-level diagnostics

Q5. What is the MLP for this launch?

The MLP will deliver automated cross-domain diagnosis and root-cause attribution for detected issues, eliminating manual timeline reconstruction and reducing mean-time-to-resolution from hours to minutes.

At its core, the MLP will introduce a unified diagnostic ledger that automatically stitches data from multiple domains—including logs, crashes, user interactions, and service logs (detailed in Q3)—into a time-ordered

investigation context when an issue is detected. This allows developers to start with a pre-assembled diagnostic view rather than manually gathering and correlating data across multiple tools.
The system will support domain-specific analyzers and runbooks that execute directly against this shared context to correlate failures to likely root causes—such as specific code changes, configuration updates, or feature flag rollouts—without requiring manual log analysis. When analyzers determine that additional data is needed to confirm a hypothesis, the framework adaptively pulls diagnostic data from impacted devices or services, expanding investigation depth only when necessary while controlling cost and system overhead. The MLP will also introduce extensibility through data plugins and prompt-based analyzers, allowing teams to contribute their own domain signals and diagnostic logic without building bespoke pipelines.

Q6. How extensible is the plugin architecture for teams wanting to add domain-specific data and analyzers?
Extensibility is frictionless by design, requiring minimal engineering effort. Teams can add new diagnostic data sources by exposing an API or MCP endpoint and registering it with the Troubleshooting Suite's APIs. The data source must be query able using a device identifier and time range, the framework then handles the retrieval, and integration of the data into the diagnostic ledger automatically. Beyond data, teams define domain-specific analyzers without traditional code. An analyzer is expressed as a prompt, skill or steering doc describing patterns to detect, conclusions to draw, and how to present findings and can be deployed using the self-service tooling. Analyzers are first-class citizens that can:

a) **Invoke other agents** - delegate sub-tasks to specialized agents (e.g., a crash triage agent, or a connectivity agent) and compose their outputs into a unified diagnosis.
b) **Call MCPs and external tools** - pull live data, run computations, or query knowledge bases mid-analysis.
c) **Chain reasoning steps** - express complex diagnostic logic as sequenced prompt-driven reasoning

This creates a federated analyzer model where domain teams extend diagnostics while Troubleshooting Suite provides shared context and execution layer.

Example Analyzer Prompt:

```
"For this DSN <DSN>, check for network degradation within 5 minutes before the
crash. If present, run traceroute via NetworkDiagnosticsAgent, correlate with
recent config changes using ConfigManagementMCP, and return root cause with
confidence score."
```

Q7. What factors determine how quickly end-to-end diagnosis and root-cause attribution can be completed?
Diagnosis time is a function of two variables: where the diagnostic data lives and how complex the failure pattern is. It is not a fixed SLA.

a) **Data availability is the largest factor:**
    a. Cloud-side data (service logs, metrics, crash reports, ticket data) is available within seconds to minutes; it is continuously ingested and readily accessible.
    b. Device-side data (detailed logs, system traces) requires on-demand retrieval from the device. Timing depends on device availability, connectivity, and upload duration.
b) **Failure complexity is the second factor:** Simple failures that match known patterns are attributed quickly, while complex cross-domain issues may require additional data collection and deeper analysis before attribution confidence reaches the threshold for action.

The Troubleshooting Suite uses an event-driven architecture where the diagnostic ledger is constructed incrementally as data becomes available. As soon as new data is added, analyzers automatically re-run and update the diagnosis, enabling progressive attribution, so teams begin investigation with early insights while the system continues refining the root cause as additional data is collected.

Q8. Once an issue is detected, how will the on-demand diagnostic ledger get built?

When an issue is detected, the Troubleshooting Suite automatically initiates diagnostic data collection. The system constructs a time-ordered diagnostic ledger by stitching data across domains (logs, crashes, user interactions, service events, and more) into a unified view of the impacted device. Unlike static snapshots, the ledger updates continuously as the new data accumulates. The event-driven architecture ensures that every update propagates downstream: active analyzers re-evaluate, and the associated diagnostic investigation summary updates automatically. The result is a living diagnostic record that gives developers an increasingly complete view of the failure as data accumulates.

Q9. How does automated root cause attribution work?

Root-cause attribution is performed by Analyzers—domain-specific components that execute against the full investigation context for an issue. The key enabler is the unified cross-domain context (diagnostic ledger) that eliminates the need for each team to build and maintain their own correlation logic.

Systems like Photon, Logzilla, LEDA and others that hosts analyzers can integrate with Troubleshooting Suite and consume the unified diagnostic context, share their diagnostic skills, knowledge bases, and contribute their mitigation and root cause attribution capabilities—creating a two-way value exchange.

a) **Photon (DS2) –** It hosts analyzers in the form of particles that codify diagnostic steps and is used across all product lines. It is used by support engineers for diagnosis and remediation of customer-reported issues received via calls and chats.
b) **Logzilla (previously owned by FireTV, now DS2) -** A log parser and analyzer tool used by developers, primarily for FireTV devices.
c) **LEDA - Log Evaluation and Debugging Assistant (Alexa)** – An AI-powered agentic triaging system that supports logs, crashes and metrics analysis backed by automatically generated runbooks and context from source code. Used for diagnosis of customer-reported issues via Report-A-Bug (RAB) flow, and for tickets created in JIRA/SIM for EFDs. [wiki link]

All these systems carry out analysis on common underlying datasets—primarily logs, to perform diagnosis for issues across different product lines. With this launch, these systems will no longer be required to directly extract device logs or crash stack traces; instead, they can trigger analysis against the diagnostic ledger itself, leveraging the shared context to improve efficiency and consistency across all troubleshooting workflows. We are already partnering with LEDA on a POC where its analyzers will run against the shared diagnostic context on the Panacea framework for systematically detected issues.

Q10. What are your hurdles/challenges for this to be successful?

Delivering automated cross-domain diagnosis at scale introduces several technical and organizational challenges that must be addressed for the system to succeed.

a) **Domain expertise is tribal and fragmented across teams** - Domain knowledge exists informally across teams rather than in automated systems, limiting attribution accuracy. We will partner with domain teams to encode their expertise as analyzers and runbooks, establishing feedback loops where developers validate and refine results.
b) **Limited availability and programmatic access to diagnostic data** - Automated diagnosis requires timely access to data across devices, services, and applications, but some data is programmatically inaccessible or depends on device availability. We will establish standardized APIs and MCP integrations for programmatic access, prioritize service-side diagnostics, and progressively expand evidence collection as the event-driven architecture updates the ledger incrementally.

c) **Heavy onboarding effort prevents federated scaling** - If contributing data and analyzers require significant engineering effort, the federated model will not scale. We will provide plugins-based data extensibility, that allow teams to contribute data and analyzers.
d) **Slow or inaccurate attribution undermines developer trust -** Developers will revert to manual investigation if diagnosis is slow, frequently incorrect, or lacks supporting evidence. We will integrate with developer workflows to quickly surface insights, provide confidence scores with transparent evidence, and allow developers to validate conclusions.

Addressing these challenges requires strong governance, measurable impact metrics (MTTR reduction, higher first-attempt resolution, lower call AHT), and continuous feedback to improve attribution accuracy and onboarding speed.

Q11. How does the system handle data collection without impacting device performance?
The diagnostic ledger is constructed on-demand, retrieving only necessary data from existing domain sources when an issue is detected, or an investigation is initiated. Investigations begin with lightweight diagnostic context—key logs, crash traces, and metadata already generated during normal operation. Additional data is collected only when needed to increase attribution confidence, using targeted sampling from impacted device cohorts rather than the entire fleet.
Correlation and analysis occur off device within the Troubleshooting Suite framework, ensuring device runtime performance is unaffected. For certain issue types where device cohorts exhibit anomalous behavior, the system may prioritize cloud-side service diagnostics to prevent exacerbating problems on already-stressed devices. The decision to expand device-side data collection depends on the nature of the issue, available cloud-side signals, and device health metrics. This approach protects device health while keeping storage and compute costs predictable.

Q12. How does the system handle privacy when analyzing device-specific data?
The system uses aggregated signals for issue detection and cohort analysis, reserving device-specific data for detailed investigation only when necessary and only by authorized users. Rather than exposing raw data, the system surfaces correlated insights that highlight failure patterns and root causes. This correlation layer acts as a privacy-preserving abstraction, giving developers answers about what happened and why without requiring direct access to raw data across multiple domains.

Q13: What are the headlines we dread?
The following headlines represent scenarios we must prevent:

“Automated diagnosis misattributes root cause — teams spend days fixing the wrong issue.” Incorrect attribution would erode developer trust and prolong incident resolution. Troubleshooting Suite mitigates this by relying on domain analyzers and runbooks authored by domain experts, combined with progressive diagnosis as new evidence arrives. Attribution results include supporting evidence and confidence scores, so engineers can quickly validate conclusions.

“Automated diagnosis creates alert fatigue — developers begin ignoring notifications" 247
If data quality degrades and false correlations overwhelm the system with diagnostic noise, 248 developers will spend more time filtering through irrelevant insights than they would have spent 249 on manual investigation, leading to notification fatigue and system abandonment.

“Automated diagnosis violates customer privacy” Expanding cross-domain visibility must not compromise data privacy or access boundaries. Troubleshooting Suite preserves existing source system access controls, enforces

role-based access, and separates aggregated cohort insights from device level diagnostics to ensure sensitive data is only visible to authorized users.

DEVELOPER BLOG:

Diagnosing cross-domain issues has traditionally required developers across teams to gather logs from multiple systems and manually correlate data to understand what went wrong. Without a shared diagnostic context, investigations often take days or weeks before teams can begin fixing the issue. With this launch, investigation becomes automated end-to-end. The system correlates data across devices, services, and applications, enabling developers to move directly from detection to reproduction and remediation. The cross-domain diagnosis can be accessed via below integration points:

1) Troubleshooting AI Agent – natural language + autonomous actions: Developers can ask questions, retrieve data, and trigger workflows using natural language. Example prompts:

a) *"For top crashes on Galileo, what were the common failure patterns before the spike?",*
b) *"Analyze this Jira ticket?"*

The agent correlates cross-domain diagnostics and can automatically trigger log uploads, runbooks, or tickets. MCP integrations allow teams to add domain-specific context. For customer support workflows, it surfaces DSN-level insights with recommended actions to accelerate triage and reduce handling time.

2) Kiro CLI and IDE Integration with MCP servers – programmatic access to insights: Developers can integrate the troubleshooting agent for end-to-end diagnosis directly into their workflows through the Kiro CLI or IDE. Example:

```
kiro-cli --agent troubleshooting-agent "Analyze this issue <ticket>" # Analyze issue ticket
kiro-cli --agent troubleshooting-agent "What's wrong with this DSN ABC123XYZ?" # Investigate a device
```

With a single command, the system performs cross-domain analysis — adaptively collecting and correlating the necessary signals — and returns root-cause attribution in seconds; enabling seamless integration with on-call workflows, CI/CD pipelines, and release validation cycles.

3) Diagnostic Timeline Viewer – deep dive across devices: Developers can explore the diagnostic ledger, view the correlated data, and run prompt driven investigation using natural language through a user interface for:

a) DSN based look up - deep dive into a single device timeline
b) CID based look up - cross-device timeline.

4) Loom Troubleshooting Suite - ready to use investigation: When issues are detected, subscribed developers/troubleshooters receive a notification (email/ticket) linking to Loom which shows:

a) Issue meta data (software version, region, device types, etc.)
b) Correlated cross-system insights
c) Events leading to the failure, along with issue attribution to relevant code path,
d) Suggested remediation steps.

Developers start with a ready-to-use cross-system investigation instead of gathering data. If a customer's DSN is part of an impacted cohort, support agents can use the same insights to guide live troubleshooting.